考点4: 图像增强 - 随机抽取图片并进行数据增强

一.考试重点:

1. 从图库随机抽取指定比例的图片
2. 对抽取的图片进行多种增强(翻转/旋转/亮度调整等)
3. 保存增强后的图片

二.在计算机视觉任务中，数据增强（Data Augmentation） 是提升模型泛化能力的重要手段。

使用 OpenCV（Open Source Computer Vision Library）可以高效地实现多种图像增强操作。

以下是基于 OpenCV 的常用数据增强方法及对应的 Python 实现代码。

前提准备 import cv2 import numpy as np import random import os 假设你有一张图像 img （BGR 格式，由 cv2.imread() 读取）： img = cv2.imread('example.jpg') # shape: (H, W, C) ☑ 常用数据增强方法及 OpenCV 实现

1. 水平翻转（Horizontal Flip）
   def horizontal_flip(img):
   return cv2.flip(img, 1) # 1 表示水平翻转
2. 垂直翻转（Vertical Flip）
   def vertical_flip(img):
   return cv2.flip(img, 0) # 0 表示垂直翻转
3. 旋转（Rotation）
   def rotate(img, angle, scale=1.0):
   h, w = img.shape[:2]
   center = (w // 2, h // 2)
   M = cv2.getRotationMatrix2D(center, angle, scale)
   rotated = cv2.warpAffine(img, M, (w, h))
   return rotated
   可以随机选择角度，如 angle = random.uniform(-30, 30)
4. 缩放（Scaling / Resize）
   def scale(img, fx, fy):
   return cv2.resize(img, None, fx=fx, fy=fy, interpolation=cv2.INTER_LINEAR)
   如 fx=fy=1.2 表示放大 20%
5. 平移（Translation）
   def translate(img, tx, ty):
   h, w = img.shape[:2]
   M = np.float32([[1, 0, tx], [0, 1, ty]])
   return cv2.warpAffine(img, M, (w, h))
   tx，ty 为像素偏移量，可正可负

6. 仿射变换（Affine Transform）
   def affine_transform(img):
   h, w = img.shape[:2]
   pts1 = np.float32([[50, 50], [200, 50], [50, 200]])
   pts2 = np.float32([[10, 100], [200, 50], [100, 250]])
   M = cv2.getAffineTransform(pts1, pts2)
   return cv2.warpAffine(img, M, (w, h))
   可用于模拟视角变化
7. 透视变换（Perspective Transform）
   def perspective_transform(img):
   h, w = img.shape[:2]
   pts1 = np.float32([[0, 0], [w, 0], [0, h], [w, h]])
   pts2 = np.float32([[50, 50], [w - 50, 20], [30, h - 30], [w - 30, h - 30]])
   M = cv2.getPerspectiveTransform(pts1, pts2)
   return cv2.warpPerspective(img, M, (w, h))
8. 亮度/对比度调整
   def adjust_brightness_contrast(img, alpha=1.0, beta=0.0):
   alpha > 1 增加对比度，<1 减少；beta > 0 增加亮度

return cv2.convertScaleAbs(img, alpha=alpha, beta=beta) 示例： adjust_brightness_contrast(img, alpha=1.2, beta=30)

9. 高斯噪声（Gaussian Noise） def add_gaussian_noise(img, mean=0, sigma=25): noise = np.random.normal(mean, sigma, img.shape).astype(np.uint8) noisy_img = cv2.add(img, noise) return noisy_img
10. 随机裁剪（Random Crop） def random_crop(img, crop_ratio=0.8): h, w = img.shape[:2] new_h, new_w = int(h * crop_ratio), int(w * crop_ratio) y = random.randint(0, h - new_h) x = random.randint(0, w - new_w) return img[y:y+new_h, x:x+new_w] 注意：裁剪后尺寸变小，可能需后续 resize 回原尺寸 🧩 组合增强（Pipeline） 你可以将多个增强方法组合成一个 pipeline，并随机启用部分操作： def augment_image(img): if random.random() > 0.5: img = horizontal_flip(img) if random.random() > 0.5: img = rotate(img, random.uniform(-15, 15)) if random.random() > 0.5: img = adjust_brightness_contrast(img, alpha=random.uniform(0.8, 1.2), beta=random.uniform(-30, 30)) if random.random() > 0.5: img = add_gaussian_noise(img, sigma=random.uniform(0, 15)) return img ⚠ 注意事项 OpenCV 默认使用 BGR 色彩空间，若后续送入深度学习框架（如 PyTorch/TensorFlow），通常需要转换为 RGB。 所有几何变换（旋转、仿射等）可能导致图像边界出现黑边，可通过 borderMode=cv2.BORDER_REPLICATE 等参数优化。 数据增强应在训练时动态进行，验证/测试阶段一般不使用（除非做 TTA：Test-Time Augmentation）。 📚 参考 OpenCV 官方文档：https://docs.opencv.org/

    user 对某个图库里的数据随机抽取一定比例做相应的数据增强 assistant 要对某个图库（图像文件夹）中的数据随机抽取一定比例，并对这些抽中的图像进行数据增强，可以使 用 Python + OpenCV 实现一个完整的流程。下面提供一个结构清晰、可直接运行的示例代码。 ☑ 功能目标
11. 输入：一个包含图像的文件夹（如 ./images/ ）
12. 操作： 随机抽取指定比例（如 30%）的图像 对每张抽中的图像应用一组数据增强（如水平翻转 + 亮度调整 + 旋转） 将增强后的图像保存到输出目录（如 ./augmented/ ）
13. 保留原始文件名前缀（如 img_001.jpg → img_001_aug_1.jpg ） 📦 所需依赖 pip install opencv-python numpy 📄 完整代码实现 import os import cv2 import numpy as np import random import shutil from pathlib import Path

配置参数

INPUT_DIR = "./images" # 原始图像目录 OUTPUT_DIR = "./augmented" # 增强后图像保存目录 AUG_RATIO = 0.3 # 抽取比例，例如 0.3 表示 30% SEED = 42 # 随机种子（可选，用于复现）

支持的图像格式

IMG_EXTENSIONS = ('.jpg', '.jpeg', '.png', '.bmp', '.tiff')

设置随机种子（可选）

random.seed(SEED) np.random.seed(SEED)

数据增强函数（可自定义组合）

def augment_image(img):

对图像应用一系列随机增强

水平翻转（50% 概率）

if random.random() > 0.5: img = cv2.flip(img, 1)

随机旋转 [-10°, 10°]

angle = random.uniform(-10, 10) h, w = img.shape[:2] center = (w // 2, h // 2) M = cv2.getRotationMatrix2D(center, angle, 1.0) img = cv2.warpAffine(img, M, (w, h), flags=cv2.INTER_LINEAR, borderMode=cv2.BORDER_REFLECT)

亮度/对比度调整

alpha = random.uniform(0.9, 1.1) # 对比度 beta = random.uniform(-20, 20) # 亮度 img = cv2.convertScaleAbs(img, alpha=alpha, beta=beta)

可选：添加轻微高斯噪声

if random.random() > 0.7: noise = np.random.normal(0, 10, img.shape).astype(np.uint8) img = cv2.add(img, noise) return img

主流程

def main():

创建输出目录

Path(OUTPUT_DIR).mkdir(parents=True, exist_ok=True)

获取所有图像文件路径

all_files = [] for root, _, files in os.walk(INPUT_DIR): for f in files: if f.lower().endswith(IMG_EXTENSIONS): all_files.append(os.path.join(root, f)) if not all_files: print("❌ 未找到任何支持的图像文件！") return

随机抽取

num_to_aug = int(len(all_files) * AUG_RATIO) selected_files = random.sample(all_files, num_to_aug) print(f"🗀 总图像数: {len(all_files)}") print(f"📦 随机抽取 {num_to_aug} 张进行增强...")

对每张选中图像进行增强并保存

for idx, img_path in enumerate(selected_files, 1): try: img = cv2.imread(img_path) if img is None: print(f"⚠ 跳过无效图像: {img_path}") continue aug_img = augment_image(img)

构造新文件名（避免覆盖）

rel_path = os.path.relpath(img_path, INPUT_DIR) stem = os.path.splitext(rel_path)[0].replace(os.sep, '_') # 将子目录路径转为下划线 ext = os.path.splitext(img_path)[1] new_name = f"{stem}*aug*{idx:04d}{ext}" save_path = os.path.join(OUTPUT_DIR, new_name) cv2.imwrite(save_path, aug_img) print(f"[{idx}/{num_to_aug}] 已保存: {new_name}") except Exception as e: print(f"❌ 处理 {img_path} 时出错: {e}") print(f"\n☑ 数据增强完成！增强图像已保存至: {OUTPUT_DIR}")

if **name** == "**main**": main()

🔍 示例说明 假设你的目录结构如下： images/ ├── cat1.jpg ├── dog1.png ├── scene/ │ └── beach.jpg └── bird.jpeg 运行脚本后（AUG_RATIO=0.5），可能输出： augmented/ ├── cat1_aug_0001.jpg ├── scene_beach_aug_0002.jpg └── bird_aug_0003.jpeg 子目录路径会被扁平化并用 _ 连接，避免文件名冲突。 🛠 可扩展建议 多增强版本：对每张图像生成多个增强副本（循环多次调用 augment_image） 同步处理标注：如果是目标检测或分割任务，需同时增强 .txt / .json / mask 文件 使用 imghdr 或 PIL 更安全地判断图像有效性 多进程加速：对大量图像可用 concurrent.futures.ThreadPoolExecutor 如果你有具体的增强需求（如只做旋转、或需要保持标签一致性），欢迎补充说明，我可以为你定制代 码！

三.案例

1.案例一:

```
#导入opencv库，处理图片或视频
import cv2
#导入numpy库，用于数值计算，并取了个别名np
import numpy as np
#导入随机数模块
import random
#导入操作系统模块，处理文件或文件夹
import os
#从pathlib模块中导入path类
from pathlib import Path

#定义单张图片增强函数，img参数传入图片路径
def augment_image(img):
    """
    对单张图像应用随机增强
    考点：组合多种增强方法,随机应用
    """
    # 1. 生成一个0~1随机数,如果随机数大于0.5就水平翻转（50%概率）
    if random.random() > 0.5:
        #对图片进行水平翻转，第一个参数：img是需要翻转的图片，第二个参数1表示水平翻转，0表示垂直翻转，-1表示水平和垂直同时翻转
        img = cv2.flip(img, 1)

    # 2. 随机旋转 [-15°, 15°]
    angle = random.uniform(-15, 15)

    #获取图片的高度和宽度
```

```
    h, w = img.shape[:2]
    #计算图片的中心点，并存在变量中
    center = (w // 2, h // 2)
    #生成旋转矩阵，center是旋转中心，angle旋转角度，1.0绽放比例
    M = cv2.getRotationMatrix2D(center, angle, 1.0)
    #执行仿射变换
    img = cv2.warpAffine(
        img, M, (w, h),        #图片，旋转矩阵，宽高
        flags=cv2.INTER_LINEAR, #指定插值方式
        borderMode=cv2.BORDER_REFLECT  #指定边界填充方式
    )

    # 3. 亮度/对比度调整
    #随机生成对比度系数，并存在变量alpha
    alpha = random.uniform(0.8, 1.2)
    #随机生成亮度偏移值，并存在变量beta中
    beta = random.uniform(-30, 30)
    #执行亮度和对比度调整
    img = cv2.convertScaleAbs(img, alpha=alpha, beta=beta)

    # 4.(考点) 生成随机数,大于0.7就添加高斯噪声 (30%概率)
    if random.random() > 0.7:
        #生成高斯噪声
        noise = np.random.normal(0, 10, img.shape).astype(np.uint8)
        #把噪声添加到图像上
        img = cv2.add(img, noise)
    #结束函数的同时返回增强后的图片
    return img

#定义批量随机增强函数
def random_augment_images(input_dir, output_dir, aug_ratio=0.3, seed=42):
    """
    从图库随机抽取指定比例的图片进行增强
    参数:
        input_dir: 输入图片目录
        output_dir: 输出目录
        aug_ratio: 抽取比例(如0.3表示30%)
        seed: 随机种子(用于复现)
    考试重点:
        1. 随机抽样 random.sample()
        2. 文件名处理
        3. 支持多种图片格式
    """
    #输出提示
    print("=" * 70)
    print("图像数据增强程序")
    print("=" * 70)

    # 步骤1: 设置随机种子(考点: 保证结果可复现)
    random.seed(seed)
    np.random.seed(seed)


    # (考点)步骤2: 创建输出目录 (文件夹)
```

```
    Path(output_dir).mkdir(parents=True, exist_ok=True)
    #输出提示
    print(f"输出目录: {output_dir}")

    # 步骤3: 获取所有图片文件
    #存所有支持的图片后缀名
    img_extensions = ('.jpg', '.jpeg', '.png', '.bmp', '.tiff')
    #声明空列表
    all_files = []
    #遍历输入文件夹，root表示当前遍历到的文件夹路径，_表示当前文件夹下的子文件夹列表，用不到，files表示当前文件夹下的文件列表
    for root, _, files in os.walk(input_dir):
        #遍历当前文件夹下的每一个文件名
        for f in files:
            #判断当前文件是否是图片
            if f.lower().endswith(img_extensions):
                #把图片的完整路径加入列表中
                all_files.append(os.path.join(root, f))

    #判断 是否没有找到任何图片
    if not all_files:
        #输出提示
        print("错误: 未找到任何图片文件!")
        #结束函数
        return

    #打印提示
    print(f"图库中共有 {len(all_files)} 张图片")

    # 步骤4: 随机抽取指定比例
    #计算需要增强的图片数量
    num_to_aug = int(len(all_files) * aug_ratio)
    #判断计算出的增强数量是否为0
    if num_to_aug == 0:
        #保证至少投到1张图片
        num_to_aug = min(1, len(all_files))  # 至少选1张
    #从所有图片中随机抽取指定数量的图片
    selected_files = random.sample(all_files, num_to_aug)
    #输出提示
    print(f"随机抽取 {num_to_aug} 张 ({aug_ratio * 100:.1f}%) 进行增强")
    print("=" * 70)

    # 步骤5: 对每张图片进行增强
    #声明变量存成功处理图片张数
    success_count = 0
    #遍历所有被抽中的图片，idx表示当前是第几张图片，img_path当前图片路径
    for idx, img_path in enumerate(selected_files, 1):
        #异常捕获
        try:
            #读取图片
            img = cv2.imread(img_path)
            #判断图片是否读取失败

            if img is None:
```

```
                #打印提示
                print(f"[{idx}/{num_to_aug}] 跳过无效图片: {img_path}")
                #路过本次循环，继续下一次循环
                continue

            #调用单张图片增强函数，对当前图片进行随机增强，并将结果保存到变量aug_img中
            aug_img = augment_image(img)

            # 构造输出文件名(考点: 避免覆盖原文件)
            #获取图片相对于输入目录的路径
            rel_path = os.path.relpath(img_path, input_dir)
            #生成不带后缀的文件名
            stem = os.path.splitext(rel_path)[0].replace(os.sep, '_')
            #获得原图片的扩展名
            ext = os.path.splitext(img_path)[1]
            #生成增强后图片的新文件名，stem是原文件名，{idx:04d}表示编号占4位，不足4位左补0，ext是原文件后缀名。eg:cat1_aug_0001.jpg
            new_name = f"{stem}_aug_{idx:04d}{ext}"

            # 保存增强后的图片
            #生成完整保存路径
            save_path = os.path.join(output_dir, new_name)
            #保存增强后的图片
            cv2.imwrite(save_path, aug_img)
            #输出提示
            print(f"[{idx}/{num_to_aug}] {os.path.basename(img_path)} → {new_name}")
            #更新成功处理图片的数量
            success_count += 1

        #捕获异常
        except Exception as e:
            #打印信息
            print(f" [{idx}/{num_to_aug}] 处理失败: {img_path}")
            print(f"  错误信息: {e}")

    # 步骤6: 输出统计信息
    print("=" * 70)
    print(f"增强完成! 成功处理 {success_count}/{num_to_aug} 张图片")
    print(f"增强图片已保存至: {output_dir}")
    print("=" * 70)

#定义演示各种增强方法的函数
def demo_augmentation_methods():
    """
    演示各种单独的增强方法
    考点: 理解每种增强的作用和参数
    """
    #输出提示
    print("\n" + "=" * 70)
    print("常用图像增强方法演示")
    print("=" * 70)


    #创建一张测试图片
```

```
    img = np.ones((200, 300, 3), dtype=np.uint8) * 200
    #调用opencv的文字绘制函数
    cv2.putText(
        img, 'Original', (50, 100),#图片,文字内容,文字的坐标
        cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2 #使用OpenCV中内置字体
    )
    #打印信息
    print("\n1. 水平翻转 - cv2.flip(img, 1)")
    #执行图片水平翻转
    img_h_flip = cv2.flip(img, 1)
    print("   用途: 增加样本多样性,适用于无方向性的图像")

    print("\n2. 垂直翻转 - cv2.flip(img, 0)")
    #执行图片垂直翻转
    img_v_flip = cv2.flip(img, 0)
    #打印信息
    print("   用途: 模拟倒置场景")
    print("\n3. 旋转 - cv2.getRotationMatrix2D() + cv2.warpAffine()")
    #声明变量存旋转角度
    angle = 30
    #获得图片宽高，img.shape正常返回三个值对应宽度，高度，通道数，img.shape[:2]取前2个值就是宽度，高度
    h, w = img.shape[:2]
    #生成旋转矩阵
    M = cv2.getRotationMatrix2D((w // 2, h // 2), angle, 1.0)
    #根据旋转矩阵妊旋转，输出图片大小仍然是原图宽高
    img_rotate = cv2.warpAffine(img, M, (w, h))
    #输出提示
    print(f"   参数: angle={angle}° (正数逆时针,负数顺时针)")
    print("\n4. 亮度/对比度 - cv2.convertScaleAbs()")
    #调整图片亮度和对比度
    img_bright = cv2.convertScaleAbs(img, alpha=1.2, beta=30)
    #输出提示
    print("   参数: alpha>1增加对比度, beta>0增加亮度")

    print("\n5. 高斯噪声 - np.random.normal()")
    #生成高斯噪声
    noise = np.random.normal(0, 25, img.shape).astype(np.uint8)
    #将噪声添加到图片上
    img_noise = cv2.add(img, noise)
    #输出提示
    print("   参数: mean=0, sigma=25 (标准差越大噪声越强)")

    print("\n6. 缩放 - cv2.resize()")
    #对图片进行绽放
    img_scaled = cv2.resize(
        img, None, fx=1.5, fy=1.5, #要缩放的图片,目标尺寸没指定,水平缩放,垂直缩放
        interpolation=cv2.INTER_LINEAR #使用线性插值
    )
    #输出提示
    print("   参数: fx=1.5, fy=1.5 (放大1.5倍)")

    print("\n7. 平移 - cv2.warpAffine()")

    #声明平移矩阵
```

```
    M_trans = np.float32([[1, 0, 50], [0, 1, 30]])
    #根据平移矩阵移动图片
    img_trans = cv2.warpAffine(img, M_trans, (w, h))
    #输出提示
    print("  参数: tx=50, ty=30 (向右移50像素,向下移30像素)")

    print("\n" + "=" * 70)

#定义主函数
def main():
    """主函数 - 使用示例"""
    #声明三个变量分别存输入图片文件夹路径，输出文件夹路径，抽取图片比例
    INPUT_DIR = "./images"  # 输入图片目录
    OUTPUT_DIR = "./augmented"  # 输出目录
    AUG_RATIO = 0.3  # 抽取30%的图片

    #调用演示各种增强方法的函数
    demo_augmentation_methods()

    #输出提示
    print("\n" + "=" * 70)
    print("开始批量增强...")
    print("=" * 70)

    #判断输入文件夹是否不存在
    if not os.path.exists(INPUT_DIR):
        #输出提示
        print(f"输入目录不存在: {INPUT_DIR}")
        print("提示: 请将图片放在 './images' 目录下")
        print("或修改代码中的 INPUT_DIR 变量")
        #创建输入文件夹
        os.makedirs(INPUT_DIR, exist_ok=True)
        print(f"\n已创建示例目录: {INPUT_DIR}")
        print("  请添加图片后重新运行程序")
    else:
       #调用批量增强图片函数
        random_augment_images(INPUT_DIR, OUTPUT_DIR, aug_ratio=AUG_RATIO)

#python程序的入口
if __name__ == "__main__":
    #调用主函数
    main()
```

运行效果如下:

```
============================================================
常用图像增强方法演示
============================================================

1. 水平翻转 - cv2.flip(img, 1)
   用途: 增加样本多样性,适用于无方向性的图像

2. 垂直翻转 - cv2.flip(img, 0)
   用途: 模拟倒置场景

3. 旋转 - cv2.getRotationMatrix2D() + cv2.warpAffine()
   参数: angle=30° (正数逆时针,负数顺时针)

4. 亮度/对比度 - cv2.convertScaleAbs()
   参数: alpha>1增加对比度, beta>0增加亮度

5. 高斯噪声 - np.random.normal()
   参数: mean=0, sigma=25 (标准差越大噪声越强)
```

```
6. 缩放 - cv2.resize()
   参数: fx=1.5, fy=1.5 (放大1.5倍)

7. 平移 - cv2.warpAffine()
   参数: tx=50, ty=30 (向右移50像素,向下移30像素)

============================================================


============================================================
开始批量增强...
============================================================
============================================================
图像数据增强程序
============================================================
输出目录: image2
图库中共有 12 张图片
随机抽取 3 张 (30.0%) 进行增强
============================================================
```

```
增强完成！成功处理 3/3 张图片
增强图片已保存至: image2
==========                                                    ==========
PS                Focus folder in explorer (ctrl + click)     t> & D:\python3.8\python.exe c:/Users/sxwork/Deskt
op/AItest/3案例41.py

================================================================================
常用图像增强方法演示
================================================================================

1. 水平翻转 - cv2.flip(img, 1)
   用途: 增加样本多样性,适用于无方向性的图像

2. 垂直翻转 - cv2.flip(img, 0)
   用途: 模拟倒置场景

3. 旋转 - cv2.getRotationMatrix2D() + cv2.warpAffine()
   参数: angle=30° (正数逆时针,负数顺时针)
```

```
4. 亮度/对比度 - cv2.convertScaleAbs()
   参数: alpha>1增加对比度, beta>0增加亮度

5. 高斯噪声 - np.random.normal()
   参数: mean=0, sigma=25 (标准差越大噪声越强)

6. 缩放 - cv2.resize()
   参数: fx=1.5, fy=1.5 (放大1.5倍)

7. 平移 - cv2.warpAffine()
   参数: tx=50, ty=30 (向右移50像素,向下移30像素)

================================================================================

================================================================================
开始批量增强...
================================================================================
================================================================================
图像数据增强程序
================================================================================
```

```
================================================================================
输出目录: image2
图库中共有 12 张图片
随机抽取 3 张 (30.0%) 进行增强
================================================================================
[1/3] face_8.jpg → face_8_aug_0001.jpg
[2/3] 5.png → 5_aug_0002.png
[3/3] 1.png → 1_aug_0003.png
================================================================================
增强完成！成功处理 3/3 张图片
增强图片已保存至: image2
================================================================================
```

四.数据增强模块（图库批量处理）操作流程（含题目要求 + 代码简析）

1.题目要求分析

该模块要从指定文件夹中随机抽取 30% 的图像，对这些图像进行水平翻转、旋转、亮度调整等增强操作，然后将增强后的图像保存到新的文件夹中。这需要我们能够处理图像文件、进行随机抽取和图像增强操作。

2.操作流程分析

1. **新建文件**：在 VS Code 中新建一个名为image_augmentation.py的文件，用于编写图像增强的代码。
2. **创建文件夹并放入图像**：在代码同级目录下创建images文件夹，将需要增强的图像放入该文件夹。
3. **运行代码**：运行image_augmentation.py代码，程序会自动对抽取的图像进行增强处理，并将增强后的图像保存到augmented文件夹中。

3.代码简析

导入所需库：os（文件路径）、cv2（图像处理）、numpy（噪声生成）、random（随机抽取）、Path（创建文件夹）

import os

import cv2

import numpy as np

import random

from pathlib import Path

---------- 配置参数（可按需修改）----------

INPUT_DIR = "./images" # 原始图像文件夹路径（当前目录下的images文件夹）

OUTPUT_DIR = "./augmented" # 增强后图像保存路径

AUG_RATIO = 0.3 # 随机抽取比例（0.3=30%）

SEED = 42 # 随机种子（保证每次抽取结果一致，方便测试）

支持的图像格式（常见格式已包含，无需修改）

IMG_EXTENSIONS = ('.jpg', '.jpeg', '.png', '.bmp', '.tiff')

设置随机种子（固定抽取结果）

random.seed(SEED)

np.random.seed(SEED)

---------- 定义数据增强函数（核心操作）----------

def augment_image(img):

"""对单张图像进行随机增强：水平翻转+旋转+亮度调整+可选高斯噪声"""

1. 水平翻转（50%概率）：random.random()生成0-1随机数，>0.5时执行翻转

if random.random() > 0.5:

img = cv2.flip(img, 1) # 1=水平翻转，0=垂直翻转

2. 随机旋转（-10°到10°之间）

```
h, w = img.shape[:2] # 获取图像高度（h）和宽度（w）
center = (w // 2, h // 2) # 旋转中心（图像中心点）
angle = random.uniform(-10, 10) # 随机旋转角度
# 获取旋转矩阵：center=中心，angle=角度，1.0=缩放比例（不缩放）
M = cv2.getRotationMatrix2D(center, angle, 1.0)
# 执行旋转：flags=插值方式（优化画质），borderMode=边界填充（避免黑边）
img = cv2.warpAffine(img, M, (w, h), flags=cv2.INTER_LINEAR, borderMode=cv2.BORDER_REFLECT)
# 3. 亮度/对比度调整
alpha = random.uniform(0.9, 1.1) # 对比度（0.9=降低10%，1.1=提高10%）
beta = random.uniform(-20, 20) # 亮度（-20=变暗，20=变亮）
img = cv2.convertScaleAbs(img, alpha=alpha, beta=beta) # 应用调整
# 4. 可选：添加轻微高斯噪声（70%概率）
if random.random() > 0.7:
# 生成高斯噪声：mean=均值（0），sigma=噪声强度（10），形状和图像一致
noise = np.random.normal(0, 10, img.shape).astype(np.uint8)
img = cv2.add(img, noise) # 图像添加噪声
return img # 返回增强后的图像
# ---------- 主流程（批量处理图像）----------
def main():
# 创建输出文件夹：parents=True表示创建多级目录，exist_ok=True不报错
Path(OUTPUT_DIR).mkdir(parents=True, exist_ok=True)
# 收集所有图像文件路径
all_files = []
# 遍历INPUT_DIR文件夹下的所有文件（包括子文件夹）
for root, _, files in os.walk(INPUT_DIR):
for f in files:
# 判断文件后缀是否在支持的图像格式中（忽略大小写）
if f.lower().endswith(IMG_EXTENSIONS):
# 拼接完整文件路径，添加到列表
all_files.append(os.path.join(root, f))
# 如果没有找到图像，打印提示并退出
if not all_files:
```

```
print("❌ 未找到任何支持的图像文件!")
return
# 计算需要增强的图像数量（总数量×抽取比例，转为整数）
num_to_aug = int(len(all_files) * AUG_RATIO)
# 随机抽取指定数量的图像文件路径
selected_files = random.sample(all_files, num_to_aug)
# 打印统计信息：总图像数、待增强数量
print(f"🗀 总图像数: {len(all_files)}")
print(f"📦 随机抽取 {num_to_aug} 张进行增强...")
# 遍历抽取的图像，逐个增强并保存
for idx, img_path in enumerate(selected_files, 1):
try:
# 读取图像文件
img = cv2.imread(img_path)
# 如果图像读取失败（如文件损坏），跳过并提示
if img is None:
print(f"⚠ 跳过无效图像: {img_path}")
continue
# 对当前图像执行增强操作
aug_img = augment_image(img)
# 构造新文件名：保留原路径结构（用下划线替换斜杠），添加aug标记
rel_path = os.path.relpath(img_path, INPUT_DIR) # 相对于INPUT_DIR的路径
stem = os.path.splitext(rel_path)[0].replace(os.sep, '_') # 文件名（不含后缀）
ext = os.path.splitext(img_path)[1] # 文件后缀（如.jpg）
new_name = f"{stem}aug{idx:04d}{ext}" # 新文件名（如apple_aug_0001.jpg）
save_path = os.path.join(OUTPUT_DIR, new_name) # 完整保存路径
# 保存增强后的图像
cv2.imwrite(save_path, aug_img)
# 打印进度信息
print(f"[{idx}/{num_to_aug}] 已保存: {new_name}")
# 捕获异常（如文件读取失败），打印错误信息
except Exception as e:
```

print(f"❌ 处理 {img_path} 时出错: {e}")

打印完成提示

print(f"\n☑ 数据增强完成!增强图像已保存至: {OUTPUT_DIR}")

执行主流程（直接运行文件时触发）

if **name** == "**main**":

main()

- **导入库**：os用于处理文件路径，cv2用于图像处理，numpy用于生成高斯噪声，random用于随机抽取，Path用于创建文件夹。
- **配置参数**：设置原始图像文件夹路径、增强后图像保存路径、随机抽取比例、随机种子和支持的图像格式。
- **设置随机种子**：保证每次随机抽取的结果一致，方便测试。
- **定义增强函数**：augment_image函数对单张图像进行随机增强，包括水平翻转、随机旋转、亮度/对比度调整和可选的高斯噪声添加。
- **主流程**：
- - 创建输出文件夹。
  - 收集所有支持的图像文件路径。
  - 计算需要增强的图像数量，随机抽取指定数量的图像。
  - 遍历抽取的图像，读取图像，进行增强处理，构造新文件名，保存增强后的图像。
  - 捕获异常并打印错误信息。

打印完成提示。

五.考试要点总结(图像增强):

1.必背增强方法:

1. 水平翻转:
   cv2.flip(img, 1)
2. 垂直翻转:
   cv2.flip(img, 0)
3. 旋转:
   M = cv2.getRotationMatrix2D(center, angle, scale)
   img = cv2.warpAffine(img, M, (w, h))
4. 亮度/对比度:
   cv2.convertScaleAbs(img, alpha=对比度, beta=亮度)
5. 高斯噪声:
   noise = np.random.normal(mean, sigma, img.shape)
   img = cv2.add(img, noise)
6. 缩放:
   cv2.resize(img, None, fx=fx, fy=fy)
7. 平移:
   M = np.float32([[1, 0, tx], [0, 1, ty]])
   cv2.warpAffine(img, M, (w, h))

2.必须掌握的函数:

1. random.sample(list, k) # 随机抽取k个元素
2. random.seed(seed) # 设置随机种子
3. os.walk(dir) # 遍历目录
4. cv2.imread() / cv2.imwrite()# 读写图片
5. img.shape # 获取图片尺寸 (H, W, C)

3.易错点:

1. 忘记设置随机种子导致无法复现
2. 文件名冲突覆盖原文件(要加后缀如_aug)
3. 增强后图片尺寸改变(如裁剪/缩放)
4. 忘记检查 cv2.imread() 返回 None
5. borderMode参数不当导致黑边

4.加分项:

1. 支持子目录遍历: os.walk()
2. 支持多种图片格式: (.jpg, .png, .bmp等)
3. 添加进度显示: [1/10], [2/10]...
4. 异常处理: try-except捕获错误
5. 统计信息: 成功/失败数量

变种题型:

- 每张图生成N个增强版本: 循环调用augment_image()
- 只对特定类别增强: 根据文件名或标签筛选
- 增强参数可配置: 接收命令行参数
- 保持原始图片比例: 使用不同的borderMode